DEEP LEARNING — 046211
FINAL EXAM B
28/5/2022

EXAM DURATION: 3 HOURS

Instructions.
1. You can use a calculator and the 2 formula sheets (4 pages).
2. You need to answer all question parts, unless you received a special permit.
3. The weight of each question is written in the form. The total number of points is 100 points.
4. If you received a special permit (due to reserve duty) you can choose to answer only two questions out of three. In that case, the questions points will be normalized so the total number of points will remain 100.
5. Please write clearly and orderly. You can write either in English or in Hebrew.
6. You must give the full details of the solution derivation. Solutions without explanations will not be given a score, unless written otherwise.
7. Once you finish the exam, you need to use gradescope to upload your exam. In addition, you need to submit the notebook.

1. Invariance and Equivariance in Multilayer Neural Networks

Shelly started working at the "Aperture" company, at the genetic testing department. She was asked to design an Multilayer Neural Network (MNN) with the following input: DNA sequences of length d. The DNA is a sequence of bases, where each base can be one of 4 options: (C, T, G, A) . Thus, the input can be described as the following matrix: $\mathbf{X} \in \mathbb{R}^{4\times d}$, where $X[j, i]$ denotes the measured value of base concentration of the j-th base at location i.

The network should output a binary classification $y \in \{-1, 1\}$ for a specific property we wish to find. The network will be trained on samples $\left\{\mathbf{X}^{(n)}, y^{(n)}\right\}_{n=1}^{N}$, with a logistic loss function $\ell\left(y, \hat{y}\right) = \log\left(1 + e^{-y\hat{y}}\right)$.

First, we will examine a network with one hidden layer of size 4×d and a LeakyReLU activation φ :

$$f_{\mathbf{w}}(\mathbf{X}) = \sum_{r=1}^{4}\sum_{k=1}^{d} W_2[r, k]\varphi(\sum_{j=1}^{4}\sum_{i=1}^{d} W_1[r, k, j, i]X[j, i]) \,,$$

where $\mathbf{w} = \{\mathbf{W}_1, \mathbf{W}_2\}$ are the layers of the weight tensors. After training is done, the classification will be done with $\text{sign}(f(X))$.

1. Which invariances exist in the network's parameters?

2. Now, Shelly noticed that the direction in which the DNA is scanned is arbitrary. Thus, if for two inputs $\mathbf{X}, \tilde{\mathbf{X}}$ we have $\forall i, j : \mathbf{X}[j, i] = \tilde{\mathbf{X}}[j, d-i+1]$, then the two inputs are equivalent in their meaning. What constraints should we put on the network's parameters to improve the network's classification performance? Explain why using an invariant hidden layer is not optimal.

3. Lastly, Shelly noticed that the measurement process in noisy, so each sample $\mathbf{X}^{(n)}$ is multiplied by an arbitrary scale. Thus if for two inputs $\mathbf{X}, \tilde{\mathbf{X}}$ we have $\forall i, j : \mathbf{X}[j, i] = c\tilde{\mathbf{X}}[j, i]$, for some constant $c > 0$, then the two inputs are equivalent in their meaning.

(a) For the given network, that is already trained, what is the effect of the scale c on the classification result?

(b) Can the arbitrary scale hurt the training process? Hint: think what happens to the gradient of each sample.

(c) How can use this information to improve the classifier performance?

2. Efficient Differentiation

We wish to optimize a loss function $\mathcal{L}(\mathbf{w})$ for $\mathbf{w} \in \mathbb{R}^d$ using Gradient Descent (GD) with some step size schedule η_t

$$\forall t = 1, 2, .. : \mathbf{w}(t) = \mathbf{w}(t-1) - \eta_t \nabla \mathcal{L}(\mathbf{w}(t-1)) \tag{2.1}$$

initialized from some $\mathbf{w}(0)$. We would like to learn the best step size schedule using GD.

1. Suppose we can consider each η_t as a separate parameter for each t. We initialize this parameter with η_0 and update η_{t-1} with a GD step on $\mathcal{L}(\mathbf{w}(t-1))$

$$\eta_t = \eta_{t-1} - \alpha_t \frac{\partial \mathcal{L}(\mathbf{w}(t-1))}{\partial \eta_{t-1}} \tag{2.2}$$

for every step of eq. 2.1, where α_t is the another step size.

Calculate $\partial \mathcal{L}(\mathbf{w}(t-1)) / \partial \eta_{t-1}$ as a function of the loss gradients $\nabla \mathcal{L}(\mathbf{w}(t-1))$ and $\nabla \mathcal{L}(\mathbf{w}(t-2))$.

2. Now we wish to update (η_{t-1}, η_{t-2}) by doing a GD step on $\mathcal{L}(\mathbf{w}(t-1))$

$$(\eta_{t+1}, \eta_t) = (\eta_{t-1}, \eta_{t-2}) - \alpha_t \left(\frac{\partial \mathcal{L}(\mathbf{w}(t-1))}{\partial \eta_{t-1}}, \frac{\partial \mathcal{L}(\mathbf{w}(t-1))}{\partial \eta_{t-2}} \right) \tag{2.3}$$

every two steps of eq. 2.1.

Calculate the derivative $\frac{\partial \mathcal{L}(\mathbf{w}(t-1))}{\partial \eta_{t-2}}$ as a function of η_{t-1},$\{\nabla \mathcal{L}(\mathbf{w}(t-k))\}_{k=1}^{3}$ and $\nabla^2 \mathcal{L}(\mathbf{w}(t-2))$.

3. Now we wish again to update $(\eta_t, \eta_{t+1}, ..., \eta_{t+T})$ by doing a GD step on $\mathcal{L}(\mathbf{w}(t-1))$ every T steps of eq. 2.1

$$(\eta_{t+T}, ..., \eta_t) = (\eta_{t-1}, ..., \eta_{t-1-T}) - \alpha_t \left(\frac{\partial \mathcal{L}(\mathbf{w}(t-1))}{\partial \eta_{t-1}}, ..., \frac{\partial \mathcal{L}(\mathbf{w}(t-1))}{\partial \eta_{t-1-T}} \right) \tag{2.4}$$

Calculate the derivative $\frac{\partial \mathcal{L}(\mathbf{w}(t-1))}{\partial \eta_{t-\tau}}$ as a function of $\left\{\eta_{t-k}, \nabla^2 \mathcal{L}(\mathbf{w}(t-k-1))\right\}_{k=1}^{\tau-1}$, $\nabla \mathcal{L}(\mathbf{w}(t-1))$ and $\nabla \mathcal{L}(\mathbf{w}(t-\tau-1))$.

4. Compare this approach (eq. 2.4 with $T > 1$) to the first one (eq. 2.2). Name one advantage for each approach. Hints: Think of computional complexity, ease of optimization, suitability of the objective.

3. "Typical" Generalization in Multilayer Neural Networks

We examine a "student" neural network $f_{\mathbf{w}}(\mathbf{x})$ with parameter vector $\mathbf{w} \in \mathbb{R}^k$ and input $\mathbf{x} \in \mathbb{R}^{d_0}$ used in a binary classification problem where the training set is $\mathcal{S} = \left\{\mathbf{x}^{(n)}\right\}_{n=1}^{N}$ sampled i.i.d. from P_X, where the binary (± 1) labels are generated by a "teacher" neural network $f_{\mathbf{w}_\star}(\mathbf{x})$ with the same architecture. To understand the "typical" generalization of the student, we examine the following "Guess and Check" algorithm to learn its weights: we randomly sample parameters vectors $\mathbf{w}_1, \mathbf{w}_2, ...$ i.i.d. from P_W, in which each parameter is sampled independently from a uniform distribution over $Q = \{-(q-1)/2, \ldots, -1, 0, 1, \ldots, (q-1)/2\}$ quantization levels, where $q = |Q|$ is an odd positive number. We do this until a stopping time t in which we perfectly fit the dataset: $\forall n : f_{\mathbf{w}_t}\left(\mathbf{x}^{(n)}\right) = f_{\mathbf{w}_\star}\left(\mathbf{x}^{(n)}\right)$. We examine a two-layer neural network with d_1 hidden neurons

$$f_{\mathbf{w}}(\mathbf{x}) = \operatorname{sign}\left(\mathbf{w}_2^\top [\mathbf{W}_1 \mathbf{x}]_+\right)$$

where $[\cdot]_+$ is the ReLU activation function, the teacher has at most $d_1^\star < d_1$ non-zero neurons (i.e., the other $d_1 - d_1^\star$ hidden neurons in the teacher to have all the incoming and outgoing weights equal to zero).

1. Calculate the probability $P_{\mathbf{w} \sim P_W}(\mathbf{w} = \mathbf{w}_\star)$.

2. Prove that

$$p_\star \triangleq P_{\mathbf{w} \sim P_W}\left(\forall \mathbf{x} : f_{\mathbf{w}}(\mathbf{x}) = f_{\mathbf{w}_\star}(\mathbf{x})\right) \geq q^{-d_0 d_1^* - d_1}. \tag{3.1}$$

3. Show that for any constant $T > 0$, we can bound the probability of stopping time $t > T$ as

$$\lfloor T \rfloor \leq \frac{\log P(t > T)}{\log(1 - p_\star)}. \tag{3.2}$$

4. Prove the generalization bound

Theorem 1. *With probability* $(1-\eta)(1-\delta)$,

$$\epsilon < \frac{(d_0 d_1^\star + d_1)\log q + \log \frac{1}{\delta} + \log\log \frac{1}{\eta}}{N} \tag{3.3}$$

Hint: Combine the results from previous sections, using the approximations $\lfloor T \rfloor \approx T$ and $\log(1 - p_\star) \approx -p_\star$ (treat these approximations as exact), and the following basic generalization bound (which we learned in class)

Theorem 2. *For any* $f \in |\mathcal{F}|$, *with probability* $1 - \delta$,

$$\epsilon \triangleq \mathbb{P}_{\mathbf{x}}\left(f_{\mathbf{w}}(\mathbf{x}) \neq f_{\mathbf{w}_\star}(\mathbf{x})\right) < \frac{\log|\mathcal{F}| + \log \frac{1}{\delta}}{N}. \tag{3.4}$$

5. Is the bound in eq. 3.3 better than the bound in eq. 3.4 in which $\mathcal{F} = \left\{f_{\mathbf{w}} : \mathbf{w} \in Q^k\right\}$ is the student hypothesis class (in which each parameter can have one of q values)? Explain and ignore the (negligble) $\log\log \frac{1}{\eta}$ term.

Question 1 Solution

See solution to question 3 in Exam A of Spring 2021.

Question 2 Solution

1. Using the chain rule, we have

$$\begin{aligned}\frac{\partial \mathcal{L}\left(\mathbf{w}\left(t-1\right)\right)}{\partial \eta_{t-1}} &= \frac{\partial \mathcal{L}\left(\mathbf{w}\left(t-1\right)\right)}{\partial \mathbf{w}\left(t-1\right)}\frac{\partial \mathbf{w}\left(t-1\right)}{\partial \eta_{t-1}} \\ &= -\nabla \mathcal{L}\left(\mathbf{w}\left(t-1\right)\right)\cdot\frac{\partial}{\partial \eta_{t-1}}\left(\mathbf{w}\left(t-2\right)-\eta_{t-1}\nabla \mathcal{L}\left(\mathbf{w}\left(t-2\right)\right)\right) \\ &= -\nabla \mathcal{L}\left(\mathbf{w}\left(t-1\right)\right)\cdot\nabla \mathcal{L}\left(\mathbf{w}\left(t-2\right)\right)\end{aligned}$$

2. In this case, we have

$$\begin{aligned}\frac{\partial \mathcal{L}\left(\mathbf{w}\left(t-1\right)\right)}{\partial \eta_{t-2}} &= \frac{\partial \mathcal{L}\left(\mathbf{w}\left(t-1\right)\right)}{\partial \mathbf{w}\left(t-1\right)}\frac{\partial \mathbf{w}\left(t-1\right)}{\partial \mathbf{w}\left(t-2\right)}\frac{\partial \mathbf{w}\left(t-2\right)}{\partial \eta_{t-2}} \\ &= \nabla \mathcal{L}\left(\mathbf{w}\left(t-1\right)\right)\frac{\partial\left(\mathbf{w}\left(t-2\right)-\eta_{t-1}\nabla \mathcal{L}\left(\mathbf{w}\left(t-2\right)\right)\right)}{\partial \mathbf{w}\left(t-2\right)}\frac{\partial\left(\mathbf{w}\left(t-3\right)-\eta_{t-2}\nabla \mathcal{L}\left(\mathbf{w}\left(t-3\right)\right)\right)}{\partial \eta_{t-2}} \\ &= -\nabla \mathcal{L}\left(\mathbf{w}\left(t-1\right)\right)\left(\mathbf{I}-\eta_{t-1}\nabla^2 \mathcal{L}\left(\mathbf{w}\left(t-2\right)\right)\right)\nabla \mathcal{L}\left(\mathbf{w}\left(t-3\right)\right)\end{aligned}$$

3. In this case, we have

$$\begin{aligned}\frac{\partial \mathcal{L}\left(\mathbf{w}\left(t-1\right)\right)}{\partial \eta_{t-\tau}} &= \frac{\partial \mathcal{L}\left(\mathbf{w}\left(t-1\right)\right)}{\partial \mathbf{w}\left(t-1\right)}\frac{\partial \mathbf{w}\left(t-1\right)}{\partial \mathbf{w}\left(t-2\right)}\frac{\partial \mathbf{w}\left(t-2\right)}{\partial \mathbf{w}\left(t-3\right)}\cdots\frac{\partial \mathbf{w}\left(t-\tau\right)}{\partial \eta_{t-\tau}} \\ &= -\nabla \mathcal{L}\left(\mathbf{w}\left(t-1\right)\right)\cdot\left[\prod_{t'=1}^{\tau-1}\left(\mathbf{I}-\eta_{t-t'}\nabla^2 \mathcal{L}\left(\mathbf{w}\left(t-t'-1\right)\right)\right)\right]\nabla \mathcal{L}\left(\mathbf{w}\left(t-\tau-1\right)\right)\end{aligned}$$

4. Advantages of the first approach (one advantage is enough for getting a full score):
   (1) It updates the step size more frequently, which is good since delayed updates can harm performance, as we learned in class.
   (2) It is cheaper, since it does not include the Hessians. As we learned in class, while calculating Hessians is extremely expensive, Hessian-vectors products can be calculated in practice, but are still more expensive then regular gradients.
   (3) The second approach requires Backpropagation through a long optimization process. This is expensive (since we have to store in memory all the optimization process), and might also lead to instabilities due to vanishing or exploding gradients.

   Advantages of the second approach:
   (1) We update the step size according the loss at a later time which is closer to what we want to minimize (the loss at the end of training), while the first approach is greedy and aims to minimize the loss after a single step.

Question 3 Solution

1. To get a specific weights we set each of the k weights to recieve a specific value out of q possible value. Since the weights are i.i.d this entails

$$P_{\mathbf{w}\sim P_W}\left(\mathbf{w}=\mathbf{w}_\star\right)=q^{-k}\,.$$

2. In this case, we need to sample correctly all the input and output weights of the $d_1^\star$ hidden neurons in the teacher neural network. In addition we need to sample $'0'$ in the output weights of all the $(d_1-d_1^\star)$ student neurons that are not in the teacher neurons. Therefore, we get

$$p_\star \geq q^{-d_0 d_1^\star - d_1}\,.$$

3. If $t>T$ it means we were not able to fit the training set for T times,

$$P\left(t>T\right)=\left(1-P_{\mathbf{w}\sim P_W}\left(\forall \mathbf{x}\in\mathcal{S}: f_{\mathbf{w}}\left(\mathbf{x}\right)=f_{\mathbf{w}_\star}\left(\mathbf{x}\right)\right)\right)^T\leq\left(1-p_\star\right)^T$$

where in the inequality we used the fact the probability of achieving a perfect classification on the training set can be lower bounded by the probability of achieving the same function as the teacher. Taking the log of both sides of the inequality we get

$$\log\left(1-p_\star\right)^T\geq\log P\left(t>T\right)\Rightarrow T\log\left(1-p_\star\right)\geq\log P\left(t>T\right)\overset{\log(1-p_\star)<0}{\Rightarrow}T\leq\frac{\log P\left(t>T\right)}{\log\left(1-p_\star\right)}\,.$$

4. Denoting $\eta\triangleq P\left(t>T\right)$, we have that $1-\eta=P\left(t<T\right)$. So, with probability $1-\eta$, we have $f_{\mathbf{w}_t}\in\mathcal{F}_T=\{f_{\mathbf{w}_1},...,f_{\mathbf{w}_T}\}$, and

$$\begin{aligned}\log\left|\mathcal{F}_T\right|=\log T&\leq\log\left(\frac{\log P\left(t>T\right)}{\log\left(1-p_\star\right)}\right)=\log\left(\frac{\log\eta}{\log\left(1-p_\star\right)}\right)\\&\approx\log\left(\frac{\log\eta}{-p_\star}\right)=\log\log\frac{1}{\eta}-\log p_\star\leq\log\log\frac{1}{\eta}+\left(d_0d_1^\star+d_1\right)\log q.\end{aligned}$$

where in the first inequality we used eq. 3.2 and in the last equality we used eq. 3.1. Plugging this into the generalization bound (eq. 3.4), we get the Theorem we aimed to prove.

5. With q quantization levels, and $k=d_0d_1+d_1$ parameters we get in eq. 3.4

$$\log\left|\mathcal{F}\right|=k\log q=\left(d_0d_1+d_1\right)\log q$$

This is larger then $\left(d_0d_1^\star+d_1\right)\log q$ we if $d_1^\star<d_1$ as we assume, and therefore leads to a worse generalization bound than in eq. 3.3.

Winter 2024 – Moed B – Question 1 Solution

1. The first is a permutation symmetry in the neurons of the hidden layer. I.e., for each two index pairs $(i,j),(k,r)$ the network's output won't change if we perform the following changes:

$$W_1[r,k,:,:] \leftrightarrow W_1[i,j,:,:] \text{ and } W_2[r,k] \leftrightarrow W_2[i,j].$$

   The second is continuous symmetry for multiplication and division by a constant i neach neuron. Thus, the network's output is invariant to the transformation:

$$\forall r,k,\ \forall c>0:\ W_1[r,k,:,:] \to c\cdot W_1[r,k,:,:] \text{ and } W_2[r,k] \to \frac{1}{c}W_1[r,k].$$

2. We want the network's output to be invariant to the symmetry described in the input. We can achieve that by making the representation learned in the hidden layer to be **equivariant**, and then sum over this representation to make the output invariant. As learned in the lectures, the representation in the hidden layer will be equivariant for a transformation if applying the transformation on the input and output of W_1 won't change the tensor. Here, we get the following constraint:

$$\forall i,j,k,r:\ W_1[k,r,j,i] = W_1[k,d-r+1,j,d-i+1].$$

In addition, to make the output of the network invariant we need the following:

$$(*)\forall r,k:\ W_2[r,k] = W_2[r,d-k+1].$$

One can think of the second layer as a layer that preserves an equivariant representation that is followed by an average pooling. That is, we can define a tensor $\tilde{W}_2 \in \mathcal{R}^{4\times d\times 4\times d}$ such that:

$$\forall i,j,k,r:\ \tilde{W}_2[j,i,r,k] = \tilde{W}_2[j,d-i+1,r,d-k+1]$$

$$W_2[k,r] = \frac{1}{4}\sum_{j=1}^{4}\sum_{i=1}^{d}\tilde{W}_2[j,i,r,k],$$

and $(*)$ is true. **Why equivariance and not invariance?** In class, we learned that we need the hidden layer to be equivariant, and only before the output we will be invariant because even if the class itself is invariant to the transformation it doesn't entail that the hidden features are necessary invariant to the transformation. Why? Let's look at the specific case of reversing the DNA order: let's assume that the hidden layer is invariant, then

$$\forall i,j,k,r: W_1[k,r,j,i] = W_1[k,r,j,d+1-i],$$

which means we are giving the same weight to input i and its complementary input $d+1-i$. Let's say there is an important feature in the form of the difference between two complementary inputs in every DNA, then we would not be able to detect it (it will be an eigenvector with eigenvalue 0 in W_1).

3. Solution:
   - (a) No effect, as $f(cX) = cf(X)$, and thus the classification doesn't change since $\text{sign}(f(cX)) = \text{sign}(f(X))$
   - (b) Yes, the optimization process could be very sensitive. The loss function of the the (n) sample is affected by the scale:

$$l(y^{(n)}, f_w(cX^{(n)})) = \log(1+\exp(-cy^{(n)}f_w(X^{(n)}))),$$

     and so does the **gradient** is affected: very small gradients for a very small c and also for for samples with very large c that are classified as true. On the other hand, we will get a very large gradient for samples that are classified as false for which c is very large. Large arbitrary differences between the gradient's magnitudes can harm the training process.
   - (c) We can normalize the input of the network, for example:

$$f_w(X) = \sum_{r=1}^{4}\sum_{k=1}^{d}W_2[r,k]\phi\left(\sum_{j=1}^{4}\sum_{i=1}^{d}W_1[r,k,j,i]\frac{X[j,i]}{\sqrt{\sum_{j'=1}^{4}\sum_{i'=1}^{d}X^2[j',i']}}\right)$$